Lab Report: 01

Course code:

CSE 2202

Course title: Algorithm Lab

Experiment Title: Insertion Sort and Selection Sort

Submitted by:

Name: Shafin Mehrab

ID: 251014058

Department:
Computer Science and Engineering (CSE)

Submitted to:

Name: **Fatema Khan**

Lecturer, University of Liberal Arts Bangladesh

Date of submission:

30-09-2026

Contents

	Page
Experiment Objectives	2
Problem Statement	2
Pseudocode	3
Trace Table	5
Time Complexity	7
Implementation	8
Discussion	9
Conclusion	9
References	9

Experiment Objectives:

- Implement Insertion Sort on numbers in Descending order and Selection Sort on characters in ascending order based on ASCII order.
- Trace immediate states and record precise operations counts (element comparisons, shifts and swaps).
- Analyze and compare best-case and worst-case time complexities.

Problem Statement:

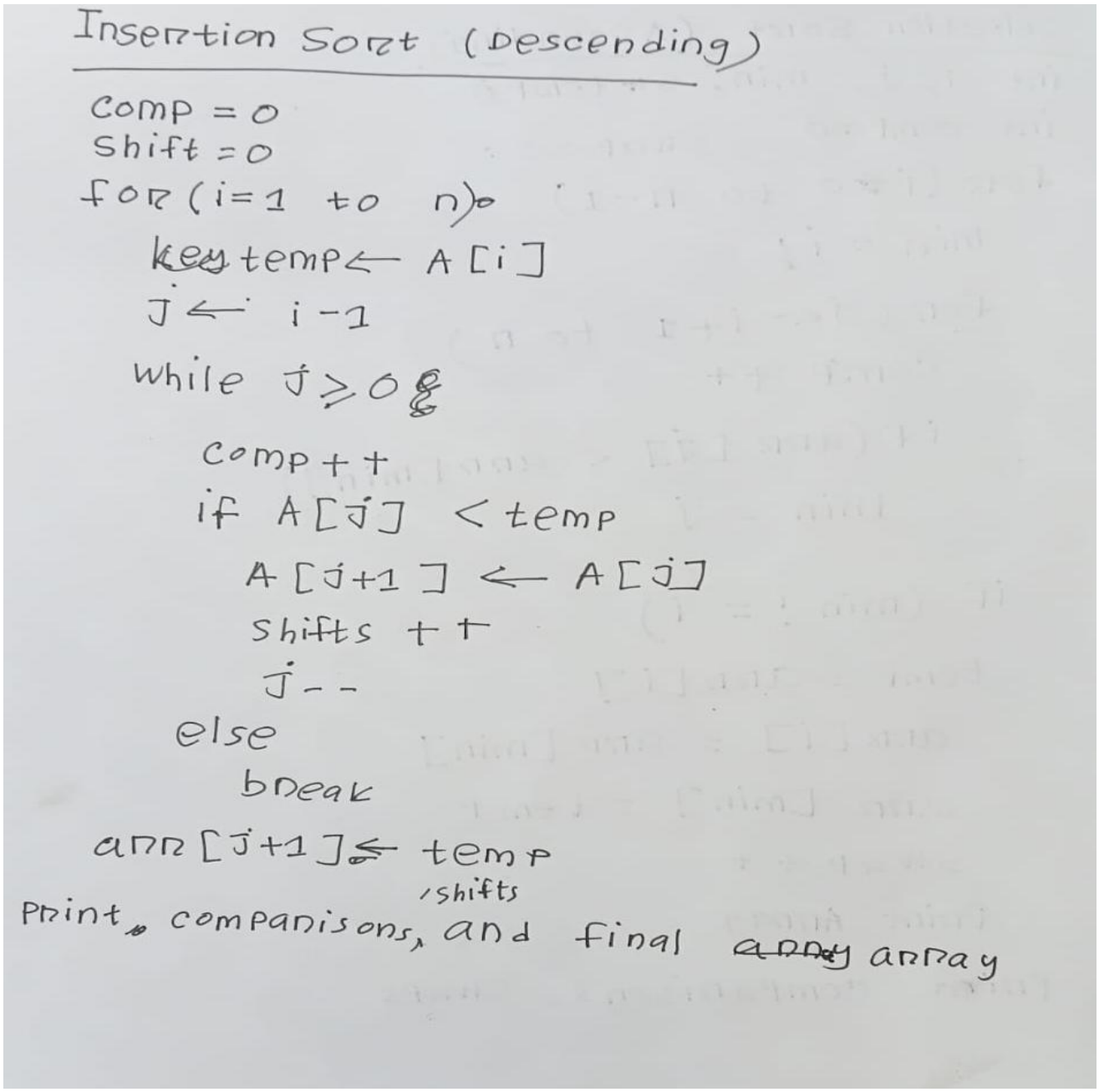
This experiment requires developing two sorting programs that take user input and display intermediate progress and final result.

Requirements:

Input	**Task 1:** Integer n ($1 \leq n \leq 100$) followed by signed integers in [-1000,1000] from standard input
	Task 2: Integer n ($1 \leq n \leq 100$) followed by n characters from standard input
Output	**Task 1:** Task 1: Array state after each insertion step (1 to n-1), total comparisons, total shifts, and final descending sorted array.
	Task 2: Array state immediately after every swap of distinct indices, total comparisons, total swaps, and final ascending sorted array.
Constraints	$1 \leq n \leq 100$; elements bounded in [-1000, 1000] for Task 1
	Uppercase and lowercase letters treated as distinct ASCII values in Task 2

Pseudocode:

Task 1:

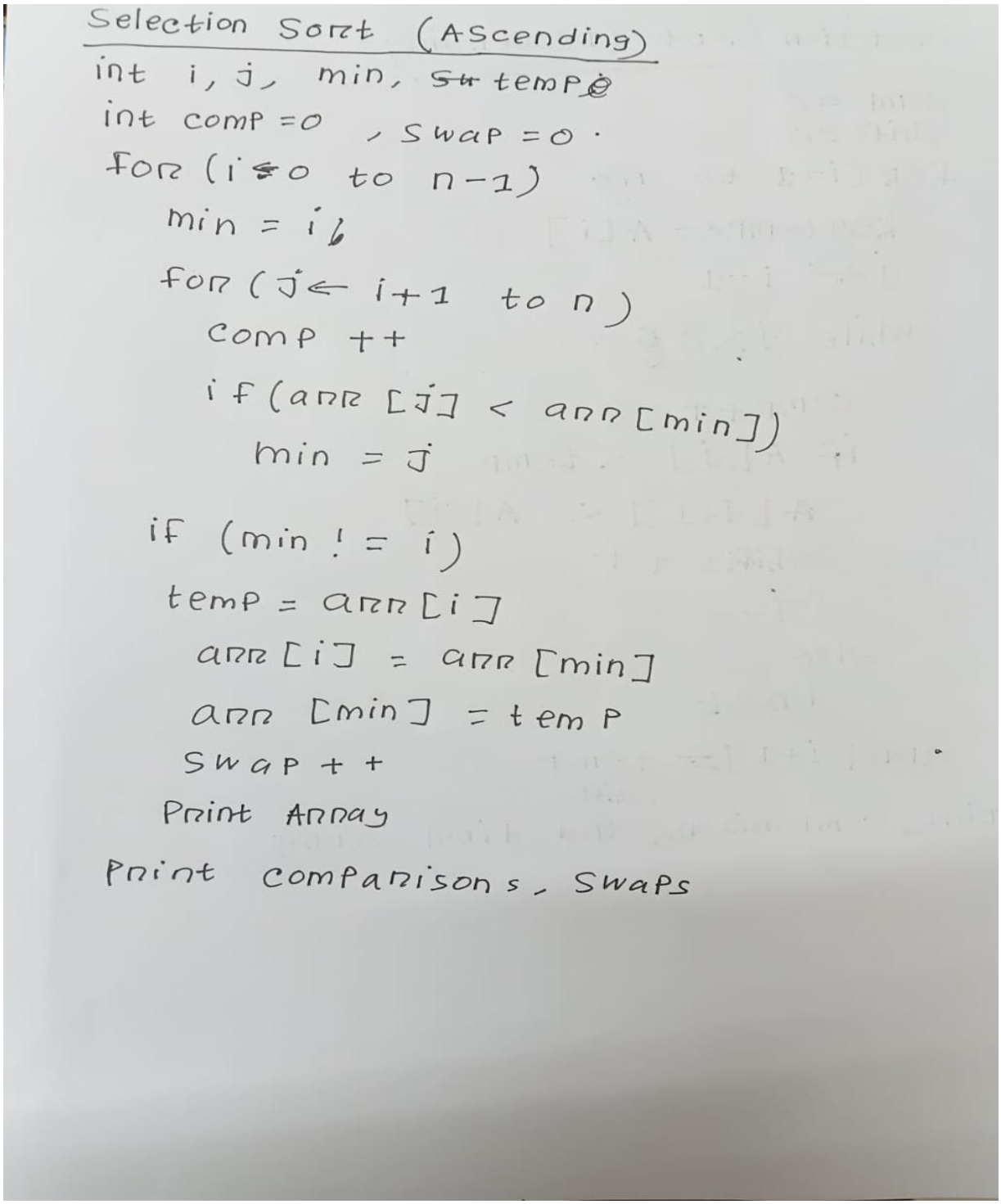


Insertion Sort (Descending)

```
comp = 0
shift = 0
for (i=1 to n)
    key temp ← A[i]
    j ← i-1
    while j ≥ 0 &
        comp++
        if A[j] < temp
            A[j+1] ← A[j]
            shifts ++
            j--
        else
            break
    arr[j+1] ← temp
```

Print, comparisons, shifts and final array

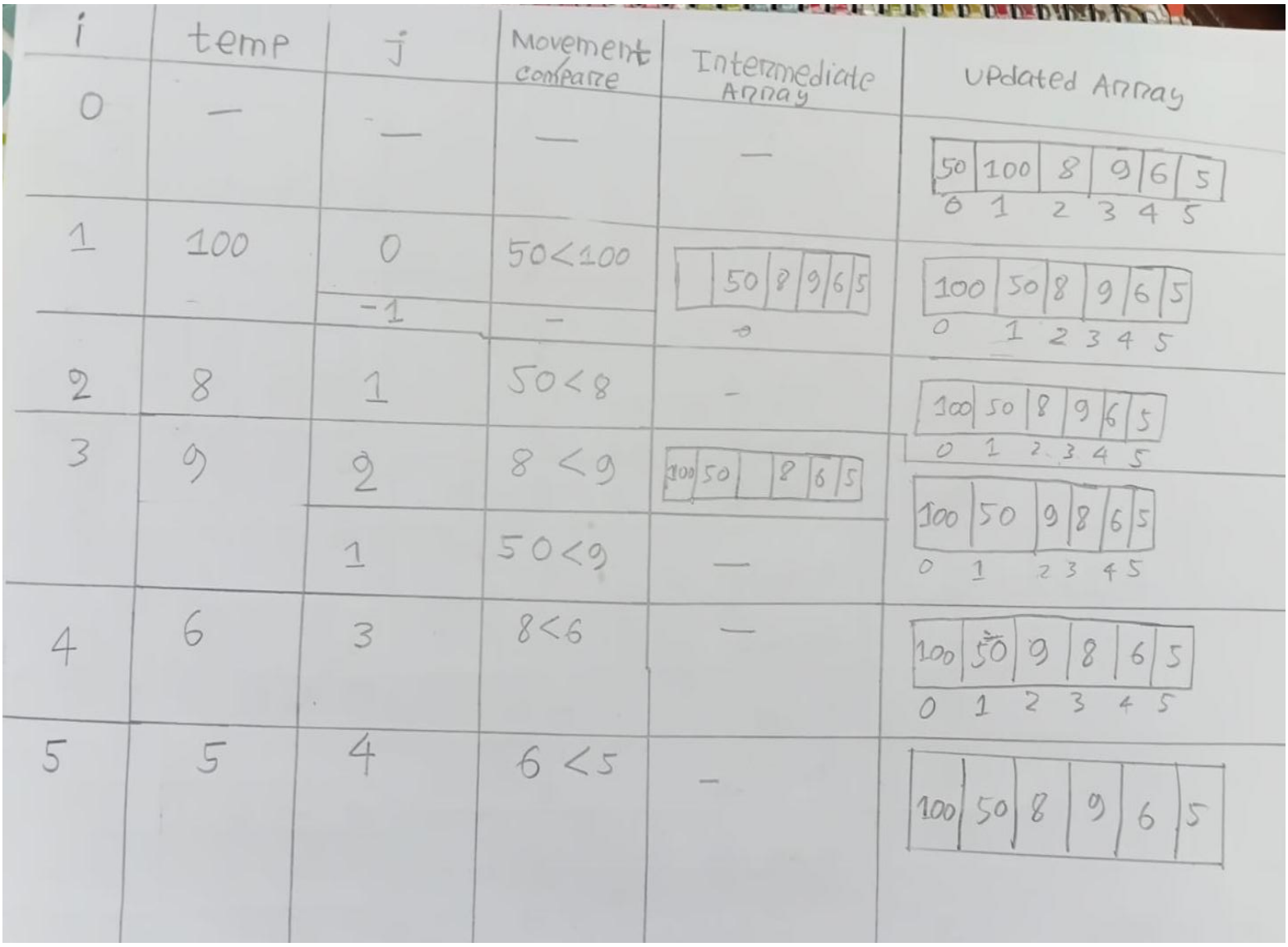
Task 2:



```
Selection Sort (Ascending)
int i, j, min, temp
int comp = 0 , swap = 0 .
for (i = 0 to n-1)
    min = i
    for (j ← i+1 to n )
        comp ++
        if (arr [j] < arr [min])
            min = j
    if (min != i)
    temp = arr [i]
    arr [i] = arr [min]
    arr [min] = temp
    swap ++
    Print Array
Print comparisons, swaps
```

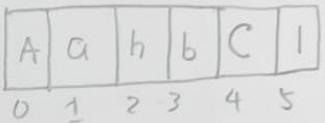
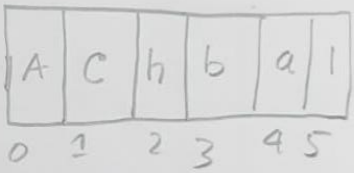
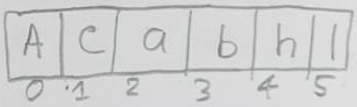
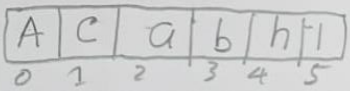
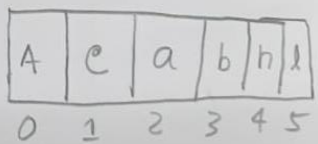
Trace Table:

Task 1:



i	temp	j	Movement / compare	Intermediate Array	Updated Array
0	—	—	—	—	50 100 8 9 6 5 (0 1 2 3 4 5)
1	100	0	50<100	50 8 9 6 5	100 50 8 9 6 5 (0 1 2 3 4 5)
		-1	—		
2	8	1	50<8	—	100 50 8 9 6 5 (0 1 2 3 4 5)
3	9	2	8<9	100 50 8 6 5	100 50 9 8 6 5 (0 1 2 3 4 5)
		1	50<9	—	
4	6	3	8<6	—	100 50 9 8 6 5 (0 1 2 3 4 5)
5	5	4	6<5	—	100 50 8 9 6 5

Task 2:

i	min	j	Comparison	UPdated Min	Swap (UPdated Array)
0	h(0)	1	a < h	1	
		2	A < a	2	
		3	b < A	2	
		4	C < A	2	
		5	l < A	2	
1	a(1)	2	h < a	1	
		3	b < a	1	
		4	C < a	4	
		5	l < C	4	
2	h(2)	3	b < h	3	
		4	a < b	4	
		5	l < a	4	
3	b(3)	4	h < b	3	
		5	l < b	3	
4	h(4)	5	l < h	4	

Time Complexity:

Task 1:

Worst Case: The outer loop runs (n-1) times. The inner loop runs i times because the array is in reverse order, preventing the break statement from executing.

Total comparisons T(n) = $\sum_{i=1}^{n-1} i$

T(n) = 1 + 2 + 3 + … + (n-1)

T(n) = $\frac{(n-1)n}{2}$ = $\frac{n^2}{2}$ - $\frac{n}{2}$

Dropping lower-order terms and constants: O (n^2)

Best Case: The outer loop runs n-1 times. The inner loop executes exactly 1 time for each iteration before the break statement is triggered.

Total comparisons T(n) = $\sum_{i=1}^{n-1} 1$

T(n) = 1 + 1 + 1 + … + 1 (evaluated n-1 times)

T(n) = n - 1

Dropping lower-order terms and constants: O(n)

Average-Case Time Complexity Calculation:

The outer loop runs n-1 times. The inner loop runs i/2 times on average.

Total comparisons T(n) = sum(i/2) from i=1 to n-1

T(n) = $\frac{1}{2} * \sum_{i=1}^{n-1} 1 = \frac{1}{2} * \frac{(n-1)n}{2} = \frac{n^2}{4} - \frac{n}{4}$

Dropping lower-order terms and constants: O (n^2).

Task 2:

Worst, Best, and Average-Case Time Complexity Calculation:

The inner loop contains no early exit conditions; it always runs to completion regardless of the initial order of the array.

The outer loop runs n-1 times. For each iteration i, the inner loop runs n - (i + 1) times.

Total comparisons T(n) = $\sum_{i=0}^{n-2}(n - i - 1)$

Substituting k = n - i - 1, as i goes from 0 to n-2, k goes from n-1 down to 1.

T(n) = $\sum_{k=1}^{n-1} k$

T(n) = 1 + 2 + 3 + ... + (n-1)

T(n) = = $\frac{(n-1)n}{2}$ = $\frac{n^2}{2}$ - $\frac{n}{2}$

Dropping lower-order terms and constants: O (n^2)

Implementation:

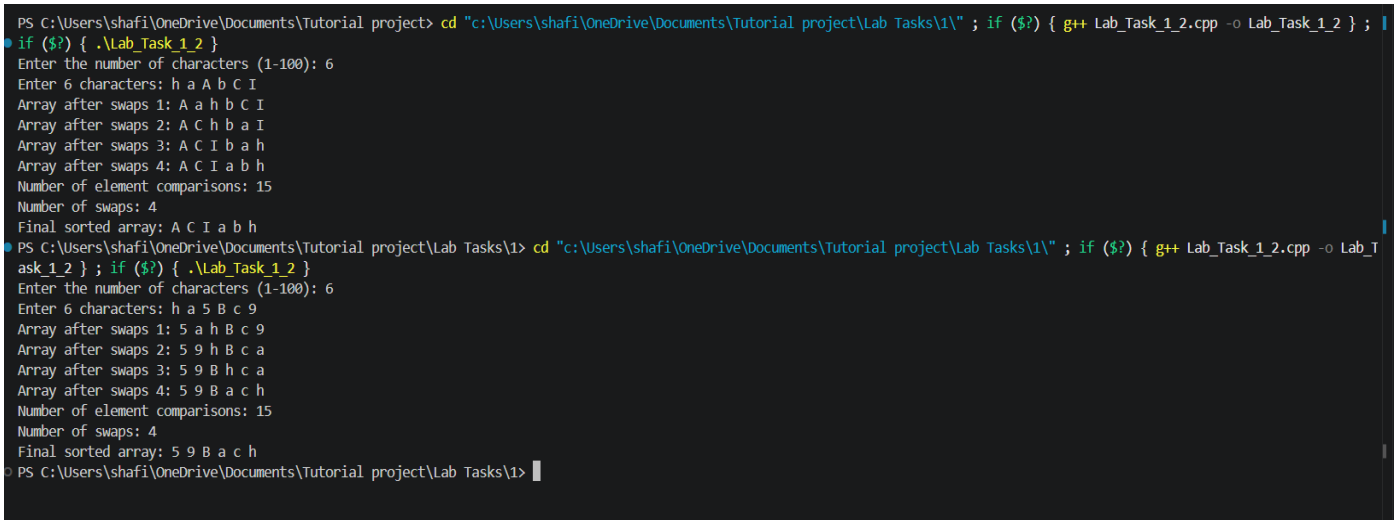
- insertionSort(int arr[], int n): Sorts integers in descending order, increments comparisons and shifts, and prints intermediate passes.
- selectionSort(char arr[], int n): Locates minimum ASCII characters, increments comparisons, and conditionally swaps and prints whenever min ≠ 1.
- printArray(): Utility that outputs the current array with uniform spacing and a trailing newline.

Testing and Results:

Task 1

```
PS C:\Users\shafi\OneDrive\Documents\Tutorial project> cd "c:\Users\shafi\OneDrive\Documents\Tutorial project\Lab Tasks\1\" ; if ($?) { g++ Lab_Task_1_1.cpp -o Lab_Task_1_1 } ;
if ($?) { .\Lab_Task_1_1 }
Enter number of elements (1-100): 6
Enter 6 integers (-1000 to 1000): 50 100 8 9 6 5
Array after insertion 1: 100 50 8 9 6 5
Array after insertion 2: 100 50 8 9 6 5
Array after insertion 3: 100 50 9 8 6 5
Array after insertion 4: 100 50 9 8 6 5
Array after insertion 5: 100 50 9 8 6 5
Number of comparisons: 6
Number of shifts: 2
Final sorted array (descending order): 100 50 9 8 6 5
PS C:\Users\shafi\OneDrive\Documents\Tutorial project\Lab Tasks\1> cd "c:\Users\shafi\OneDrive\Documents\Tutorial project\Lab Tasks\1\" ; if ($?) { g++ Lab_Task_1_1.cpp -o Lab_T
ask_1_1 } ; if ($?) { .\Lab_Task_1_1 }
Enter number of elements (1-100): 6
Enter 6 integers (-1000 to 1000): 50 100 8 6 A 5
Array after insertion 1: 100 50 8 6 0 1878003816
Array after insertion 2: 100 50 8 6 0 1878003816
Array after insertion 3: 100 50 8 6 0 1878003816
Array after insertion 4: 100 50 8 6 0 1878003816
Array after insertion 5: 1878003816 100 50 8 6 0
Number of comparisons: 9
Number of shifts: 6
Final sorted array (descending order): 1878003816 100 50 8 6 0
PS C:\Users\shafi\OneDrive\Documents\Tutorial project\Lab Tasks\1>
```

Task 2:



Discussion:

Insertion Sort is an adaptive algorithm: its comparison and shift count scale dynamically based on the input', dropping to O (n) operations when already sorted. On the other hand, Selection Sort is non-adaptive, performing $\frac{(n-1)n}{2}$ comparisons regardless of input order. However, Selection Sort minimizes data writes, performing at most n-1 memory swaps, whereas Insertion Sort performs up to O (n^2) shifts in worst-case scenarios.

Conclusion:

Both descending Insertion Sort and ASCII character Selection Sort were implemented and tested according to assignment requirements. Results proved that Insertion Sort adapts to input order with best-case O (n) time, while Selection Sort maintains a strictly quadratic O (n^2) comparison cost across all permutations. A key limitation observed is that both algorithms remain inefficient for larger input bounds (n > 1000).

References:

- No external sources used